AI Intelligence Pipeline

Technical Architecture & Scale Design Document | FrontierAtlas Team

1. Executive Summary

This document defines the production architecture for the FrontierAtlas AI Intelligence Pipeline, designed to collect, normalize, enrich, resolve, and store multi-dimensional intelligence datasets. The system targets five core verticals: startups, products, research papers, jobs, and news. Through highly concurrent ingestion, resilient multi-tier LLM fallback, and deterministic entity resolution, the pipeline converts unstructured web signals into a structured, unified entity graph.

2. Conceptual Pipeline Workflow

The ingest engine operates asynchronously, processing raw inputs through a series of stages:

Stage	Components	Primary Technology
1. Ingestion	Async crawlers, API connectors, RSS feed parsers	Python, asyncio, aiohttp
2. Rate Limiting	Adaptive token buckets, proxy pools, headers matching	Redis, Backoff + Jitter
3. Extraction	Chunking engine, structured schema validators	Pydantic, Multi-tier LLM Chain
4. Resolution	Deterministic entity resolver, alias database matcher	Regex normalizers, corporate parser
5. Storage	Relational database, entity graphs, vector indexes	PostgreSQL, Neo4j, pgvector
6. Delivery	Automated CSV exports, synced Google Sheets	Google Sheets API, python-csv

3. Horizontal Scale Strategy (500,000+ Records)

To scale the pipeline to ingest and process over 500k records without bottlenecks, the architecture shifts from a single-node sequential script to a distributed event-driven framework:

Distributed Queues: A message broker (e.g., Apache Kafka or RabbitMQ) acts as the ingestion backbone. Tasks are pushed to specific queues (e.g., *startup-scrape*, *paper-enrich*) and processed by independent, containerized worker nodes (e.g., Celery or Dramatiq) running in Kubernetes (EKS).

Horizontal Pod Autoscaling: Worker containers scale dynamically based on CPU/memory load and queue backlog depth. This guarantees that during peak periods (e.g., bulk historical crawls), ingestion rates scale without manual server tuning.

Partitioned Processing: Target URLs and API requests are distributed across workers using hash-based partitioning (e.g., partitioning by domain name) to prevent concurrent workers from hammering the same target domain simultaneously.

4. Resiliency & Rate-Limit Strategies (413s & 429s)

A. Handling 413 Payload Too Large (LLM Context Constraints)

Large raw HTML pages, PDF abstracts, or lengthy reports easily trigger 413 HTTP errors or exhaust LLM context windows. The pipeline resolves this using a **Semantic Chunking and Map-Reduce** pattern:

Windowed Chunks: The raw text is divided into standard chunks (e.g., max 12,000 characters) preserving sentence and paragraph boundaries.

Map Stage: Each chunk is processed concurrently by the LLM extraction chain to extract entity fields, descriptions, and metadata.

Reduce/Synthesis Stage: The structured outputs are aggregated, resolved, and merged into a single entity record. This dramatically minimizes payload sizes sent to LLM providers.

B. Handling 429 Too Many Requests (Adaptive Rate Limiting)

Websites and API providers (like ArXiv, GitHub, and LLM endpoints) actively enforce rate limits. The pipeline employs a multi-tiered rate limiting strategy:

Distributed Token Bucket: Workers use a centralized Redis store to track and acquire rate-limit tokens per domain/provider. This ensures global rate compliance across all distributed nodes.

Adaptive Backoff with Jitter: Requests that fail with 429 or 403 are retried using exponential backoff: $t = base^{attempt} + Jitter$, where jitter introduces randomized milliseconds to prevent synchronizing retry waves.

HTTP Header Sniffing: Scrapers inspect `Retry-After` and GitHub's `X-RateLimit-Reset` headers to pause requests dynamically until the exact reset time, preventing token exhaustions.

5. Freshness Tracking & Deduplication

For jobs and news datasets, the pipeline guarantees absolute freshness (<= 24 hours) and prevents duplicate processing across worker instances using the following mechanism:

Centralized Idempotency Key: Every crawled item is assigned a unique idempotency key: *SHA256(URL + NormalizedTitle)*. Before parsing, workers query a global Redis cache; if the key is present, the item is skipped.

Bloom Filters: To scale memory efficiency, Redis Bloom Filters are used to track billions of historically seen URLs with high lookup speed and negligible memory footprints.

UTC Normalization: Raw publication dates (including relative strings like '2 hours ago') are immediately normalized to ISO-8601 UTC format. Records older than 24 hours are discarded during the ingestion phase.

6. Unified Storage Architecture

The production pipeline stores the resolved intelligence in a hybrid storage architecture designed for multi-dimensional querying:

Database Type	Role in Architecture	Implementation Choice
Relational DB	Stores canonical entity records, jobs, news, and structured metadata. Enables transactional queries and reporting	PostgreSQL
Graph Database	Maps relationships between entities. E.g., connecting a Research Paper to its GitHub Repo, and mapping the Sta	Neo4j
Vector Database	Stores high-dimensional embeddings of research paper abstracts and product descriptions to enable semantic sim	pgvector / Pinecone